

Comparator & Functional Interfaces in Java

Quick reference notes — sorting, lambdas, and the compare() contract

1. What is Comparator?

Comparator<T> is an **interface**, not a class. It lives in `java.util` and defines one abstract method:

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

Because it has exactly **one abstract method**, it qualifies as a **functional interface** — which is why a lambda expression can be used to implement it directly. Comparator also ships with several **default** and **static** methods (like `reversed()`, `thenComparing()`, `Comparator.comparing()`) that come free once `compare()` is implemented.

Comparator vs Comparable: `Comparable<T>` is implemented *inside* the class itself via `compareTo()`, defining a single "natural ordering." Comparator is written *externally*, separate from the class being sorted, and you can define as many different Comparators as you like.

The compare() contract

```
int compare(T o1, T o2)
```

Return value	Meaning
negative	o1 comes before o2
zero	o1 and o2 are equal for ordering
positive	o1 comes after o2

2. Ways to Create a Comparator

2.1 Separate named class (implements Comparator)

A standalone class — good for reusable or complex comparison logic.

```
class Person {  
    String name;  
    int age;  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
class AgeComparator implements Comparator<Person> {  
    @Override
```

```

        public int compare(Person p1, Person p2) {
            return Integer.compare(p1.age, p2.age); // ascending by age
        }
    }

    // Usage:
    List<Person> people = new ArrayList<>();
    people.sort(new AgeComparator());

```

2.2 Anonymous inner class

Same idea, written inline without a separate named class. Verbose — mostly replaced by lambdas today.

```

Collections.sort(people, new Comparator<Person>() {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.age, p2.age);
    }
});

```

2.3 Lambda expression (modern, common way)

Since Comparator is a functional interface, a lambda can implement it directly.

```

Comparator<Person> byAge = (p1, p2) -> Integer.compare(p1.age, p2.age);

people.sort(byAge);
// or inline:
people.sort((p1, p2) -> Integer.compare(p1.age, p2.age));

```

2.4 Comparator.comparing() / method references (cleanest)

```

Comparator<Person> byAge = Comparator.comparing(p -> p.age);
people.sort(byAge);

// or directly, using a getter + method reference:
people.sort(Comparator.comparing(Person::getAge));

```

3. Chaining: reversed() and thenComparing()

```
// Sort by age, descending
people.sort(Comparator.comparing((Person p) -> p.age).reversed());

// Sort by age, then by name if ages are equal
people.sort(
    Comparator.comparing((Person p) -> p.age)
                .thenComparing(p -> p.name)
);
```

4. Where Comparator Is Used

Use case	Example
Sort a List	<code>Collections.sort(list, comparator)</code>
Sort a List (instance method)	<code>list.sort(comparator)</code>
Sort object arrays (not primitives)	<code>Arrays.sort(arr, comparator)</code>
Custom order in sorted collections	<code>new TreeMap<>(comparator) / new TreeSet<>(comparator)</code>
Custom heap ordering (common in DSA)	<code>new PriorityQueue<>(comparator)</code>
Sorting a stream	<code>stream.sorted(comparator)</code>

Example: PriorityQueue (common in DSA)

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> a - b); // subtraction style
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
```

5. Integer.compare() Instead of Subtraction

A common pattern is `(a, b) -> a - b`. This works for most values but is **unsafe** — it can silently overflow.

Note: If `a = Integer.MIN_VALUE` and `b = 1`, then `a - b` overflows and wraps around to a **positive** number, even though `a` is actually smaller than `b`. This quietly produces the wrong sort order with no exception thrown.

Integer.compare(a, b) is the safe replacement — it compares directly instead of doing arithmetic:

```
public static int compare(int x, int y) {
    return (x < y) ? -1 : ((x == y) ? 0 : 1);
}
```

Same idea applies to other wrapper types: `Double.compare()`, `Long.compare()`, `Character.compare()`, etc.

Prefer:

```
Comparator<Integer> cmp = (a, b) -> Integer.compare(a, b);
// or even simpler, using a method reference:
Comparator<Integer> cmp = Integer::compare;
```

Avoid:

```
Comparator<Integer> cmp = (a, b) -> a - b; // overflow risk
```

6. What Is a Functional Interface?

A **functional interface** is any interface with **exactly one abstract method**. That single method represents one specific behavior — exactly what a lambda expression is: an unnamed chunk of behavior.

A lambda like `(a, b) -> a - b` has no name and no declared type. Java infers what it implements from **context** — the expected interface at that spot. If that interface has only one abstract method, Java maps the lambda's parameters and body onto that method. If the interface had two or more abstract methods, Java wouldn't know which one the lambda is meant to implement — which is exactly why the "exactly one abstract method" rule exists.

Example

```
@FunctionalInterface
interface Greeter {
    void greet(String name); // one abstract method
}

Greeter g = name -> System.out.println("Hello, " + name);
g.greet("Sayan"); // prints "Hello, Sayan"
```

The @FunctionalInterface annotation

Optional, but good practice. It doesn't change behavior — it tells the compiler to **enforce** that the interface has exactly one abstract method. If a second abstract method is added by mistake later, you get a **compile error** instead of silently breaking every lambda that implements it.

Default and static methods don't count

A functional interface can have any number of default and static methods — they aren't abstract, so they don't break the "one abstract method" rule. This is exactly why `Comparator` can have `compare()` (abstract) plus `reversed()`, `thenComparing()`, `comparing()` (default/static) and still qualify:

```
@FunctionalInterface
interface Comparator<T> {
    int compare(T o1, T o2); // abstract - the "functional" method

    default Comparator<T> reversed() { // default - doesn't count
        return (a, b) -> compare(b, a);
    }

    static <T> Comparator<T> naturalOrder() { // static - doesn't count
        ...
    }
}
```

Other common built-in functional interfaces (java.util.function)

Interface	Abstract method	Use
Runnable	void run()	no args, no return
Comparator<T>	int compare(T, T)	comparison logic

Predicate<T>	boolean test(T)	true/false check, e.g. filtering
Function<T,R>	R apply(T)	transform input to output
Consumer<T>	void accept(T)	do something with input, no return
Supplier<T>	T get()	produce a value, no input

Summary

- Comparator<T> is an **interface** with one abstract method: compare(o1, o2).
- It's a **functional interface**, so it can be implemented as a separate class, an anonymous class, or (most commonly) a lambda / method reference.
- Chain with .reversed() and .thenComparing() for multi-level sorting.
- Always prefer Type.compare(a, b) over subtraction (a - b) to avoid integer overflow bugs.
- A functional interface = interface with exactly one abstract method — that's what allows a lambda to implement it directly.